

Figure 1: Big data and integration

capabilities by using *data grids* or *distributed caching* platforms. This attempts to overcome the big data challenge by increasing the integration middleware's available memory, and introducing an asynchronous link in the integration middleware's data-persistence mechanism.

How do these techniques help us overcome the challenge?

Answering that question requires a deeper understanding of the root cause of the underlying problem. A middleware solution fails at high loads due to the following issues:

- Memory overload, caused by data, threads or sockets
- Lack of system resources like threads, sockets and swap space

A distributed cache helps with the first issue, by increasing the available memory. For example, if you have ten servers with 10 GB each, a distributed cache can help you add up all that RAM and use it like local RAM, effectively giving you 100 GB. It helps with the second issue by avoiding the need for a large number of threads, or a large swap space. This is accomplished by intelligent persistence mechanisms, like write-behind-cache. In a write-behind-cache, data that needs to be persisted is written in an asynchronous manner: the write request is accepted, and the write function returns immediately. The persistence mechanism then writes into the file or DB; this frees up the persistence threads of the middleware, increasing the scalability of the overall solution.

A telecom use-case

Let us look at how the third option from the previous section, of using data grids, can be implemented. The use-case here is from a telecom scenario, and is depicted in Figure 2.

Figure 2 has conceptual similarity with Figure 1.

The *Network Switch* is the data source here. The big data processor maps to the *Mediation* solution here. The *Analytics Application* is similar to the *Data Warehouse*. The *Event Processor* system is similar to the *Fraud Management* application. Let us understand the data flow in this figure.

This is the data flow for a cell phone services provider—a telecom company, or telco, as per industry parlance.

Whenever a phone call is made, records called *Call Detail Records* are generated by the telco's hardware, and the records get collated at the *Network Switch*. The *Mediation* system then processes these records. It performs validation, filtering, etc., and gives the records to the three systems it

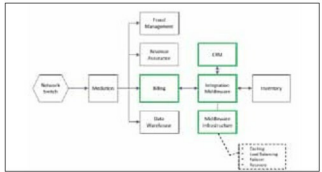


Figure 2: Big data and integration: A telecom use case

connects to: *Fraud Management*, *Billing* and *Data Warehouse*. The *Billing* system needs to connect to many other systems: *CRM*, *Inventory*, *Fraud Management*. For some of the data flows, like *CRM* to *Billing*, the volume is so high that we have to provide a direct connection from *Billing* to *CRM*, for a few use-cases (around 5 per cent result in such high volumes). Normal middleware, even with clustering and load balancing, cannot handle this. This is where the need for *Middleware Infrastructure* comes in, in the architecture (see Figure 2).

The *Middleware Infrastructure* component is a separate product that provides features like local and distributed caching, load balancing, fail-over and recovery, with much higher scalability than that provided by standard middleware products. Some of the *middleware infrastructure* products come in the form of data grids, which support scaling to hundreds of nodes. Examples of such products are Oracle Coherence, JBoss Infinispan, Websphere Extremescale and Terracotta Big Memory.

The future of Big data and integration

Any solution in the technology space starts out in a niche area, and as it becomes mainstream, it gets more and more commoditised. We can expect big data solutions to follow this path in the near future with:

1. The arrival of big data processing appliances
2. Support for big data in cloud platforms
3. Cloud-based integration platforms that are pre-packaged with middleware infrastructure.

Hopefully, this article gives you a good overview of the challenges posed to integration solutions by the advent of big data in enterprises. The example discussed, which is a use-case from the telecom domain, is generic enough to be applicable to other domains. The key value that open source brings to such solutions is that we can scale out our solution with much lower financial implications, compared to commercial solutions. **END** 

By: Prajod Vettiayattil

The author has 15 years of experience in designing and implementing business critical, scalable J2EE- and SOA-based solutions for Fortune 500 organisations. He is presently leading the open source evangelisation for Wipro's Enterprise Business Integration team. He also loves photography and reading poetry.